System & Architecture Design Document

Transport Management System — Backend API (REST + WebSocket), version 1.0.0

Document	**SDD-1.0.0** — API System & Architecture Design Document
Asset	Name **transport**, type **API**, asset code **TRANSPORT-API-001**, version **1.0.0**
System	Freight/transport management backend — 204 routes / 285 operations, 103 database tables, 20-state journey machine, Socket.IO real-time layer
Platform	Node.js ≥ 20 · Express 5 · MySQL 8 · Redis · Socket.IO v4 · PM2 · Nginx
Environments	Production `https://app.dynamicsroute.tech` · staging `https://dev.dynamicsroute.tech`
Related documents	Software Requirements Specification (SRS); API Technical Specifications (TDD/LLD) package; Request/Response package with endpoint inventory and live captures
Submitting entity	**[LEGAL NAME OF COMPANY / LICENCEE]**
Contact person	**[NAME, TITLE, EMAIL, TELEPHONE]**
Date of submission	**[DD MONTH YYYY]**

Status of this document

This design describes the system *as implemented* in the deployed service. Diagrams are generated from the mermaid sources in `sources/assets/` and rendered into this PDF together with the HTML. Fields shown in **amber** are administrative placeholders that the submitting entity completes before submission.

Contents

1	Introduction
2	Architecture overview
3	Data flow design
4	Component design
5	Data architecture
6	API design
7	Real-time architecture
8	Security architecture
9	Error handling and observability
10	Deployment and environments
11	Testing strategy
12	Design — requirements traceability
A	Appendix A — document reproduction pipeline

Figures

2.1	Layered request path — pipeline, services and stores
3.1	Level 0 context — clients, platform, infrastructure, external services
4.1	Journey status machine — execution path, decisions, terminal states

How to read this document

§2–§4 describe the static and dynamic design of one Node.js service: how a request flows through the layers, how components are packaged, and how the central journey state machine behaves. §5–§8 specify the data, API, real-time and security designs. §9–§11 cover operations: errors, logging, deployment and testing. §12 traces each design section back to the SRS requirement groups. All quantitative claims (routes, tables, statuses, message kinds) were extracted from the source and validated against a running instance.

1. Introduction

1.1 Purpose

This document specifies the system and architecture design of the Transport Management System Backend API v1.0.0 — the technical counterpart of the companion SRS. It is written so that an external reviewer can understand the architecture, the trust boundaries, the data flows and the design decisions without reading the source code, and so that a development team can maintain the system against a stable reference.

1.2 Scope

The system is a single deployable unit that serves the REST API, the Socket.IO gateway and static assets from one Node.js process, backed by MySQL and Redis, with outbound integrations to SMS, e-mail, push and payment services. Out of scope: the client applications and the internal workings of the third-party providers.

1.3 Document organization

1. **§2 Architecture overview** — style, layered request path, subsystems.
2. **§3 Data flow design** — Level 0 context diagram, sensitive-data entry points.
3. **§4 Component design** — layer contracts, package layout, the journey state machine and transition choreography.
4. **§5 Data architecture** — schema organization, keys, audit, transactions.
5. **§6 API design** — surface, envelopes, auth, validation, pagination.
6. **§7 Real-time architecture** — Socket.IO protocol, rooms, notification fan-out.
7. **§8 Security architecture** — controls end to end.
8. **§9–§11** — errors and logging, deployment, testing.
9. **§12** — traceability to the SRS.

> **Related documents**
>
> SRS-1.0.0 (requirements), the TDD/LLD package (conventions, data model, status machine, socket protocol, full endpoint inventory) and the Request/Response package (six live captures) form the rest of the submission.

2. Architecture overview

2.1 Style

The system is a **layered, service-oriented monolith**: one Node.js process serves REST, WebSocket and static traffic. Strict layering — routes → middleware → controllers → services → CRUD/SQL — keeps business rules out of the HTTP surface, while cross-cutting utilities (validator, responder, error handler, logger, transaction context) are shared. Statelessness (JWT) plus the Socket.IO Redis adapter allow horizontal scale-out without code changes.

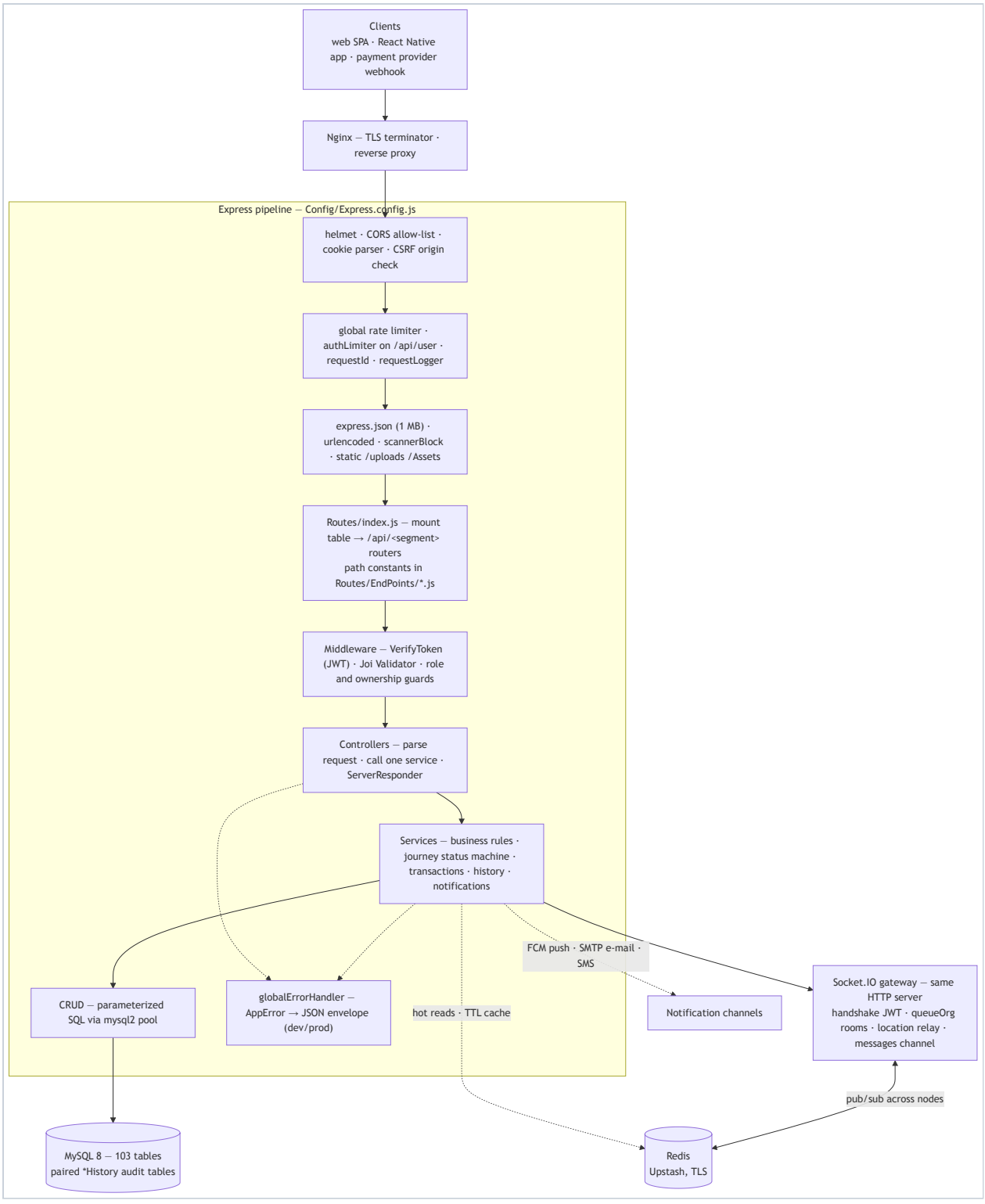

Clients
web SPA · React Native app · payment provider webhook
Nginx — TLS terminator · reverse proxy
Express pipeline — Config/Express.config.js
helmet · CORS allow-list · cookie parser · CSRF origin check
global rate limiter · authLimiter on /api/user · requestId · requestLogger
express.json (1 MB) · urlencoded · scannerBlock · static /uploads /Assets
Routes/index.js — mount table → /api/ routers
path constants in Routes/EndPoints/*.js
Middleware — VerifyToken (JWT) · Joi Validator · role and ownership guards
Controllers — parse request · call one service · ServerResponder
Services — business rules · journey status machine · transactions · history · notifications
CRUD — parameterized SQL via mysql2 pool
globalErrorHandler — AppError → JSON envelope (dev/prod)
hot reads · TTL cache
FCM push · SMTP e-mail · SMS
Notification channels
Socket.IO gateway — same HTTP server
handshake JWT · queueOrg rooms · location relay · messages channel
pub/sub across nodes
MySQL 8 — 103 tables paired *History audit tables
Redis
Upstash, TLS

Figure 2.1 — Layered request path: Nginx, Express pipeline, services, stores and the real-time gateway

2.2 Request path

```
client (web SPA / React Native / provider webhook)
  │ HTTPS JSON
  ▼
Nginx — TLS terminator · reverse proxy
  │
  ▼
Express pipeline (Config/Express.config.js)
  ├─ helmet · CORS allow-list · cookies · csrfOriginCheck
  ├─ rate limiter (global) · authLimiter (/api/user)
  ├─ requestId · requestLogger
  ├─ express.json (1 MB) · urlencoded · scannerBlock
  ├─ static /uploads · /Assets
  ├─ Routes/index.js ──▶ /api/<segment> routers
  │     ├─ VerifyToken (JWT) · Joi Validator · role/ownership guards
  │     ├─ Controller ──▶ exactly one Service ──▶ ServerResponder
  │     └─ Service ──▶ transactions · history · notifications ──▶ CRUD ──▶ MySQL
  ├─ globalErrorHandler (AppError → envelope, dev/prod rendering)
  └─ HTTP server ──▶ Socket.IO (same port, /socket.io)
                       ├─ handshake JWT · queueOrg rooms
                       ├─ location relay · messages channel
                       └─ Redis pub/sub adapter (optional)
```

2.3 Key subsystems

Subsystem	Responsibility
Identity & auth	Registration, OTP verification, JWT issue/verify, 12 roles, per-role status.
Shipper request lifecycle	Create/list/update/cancel, status machine, reconciliation, batch flows.
Dispatch — individual	Driver offers, bids, accept/reject/no-answer, shipper selection.
Dispatch — company	Company bids, vehicle-slot assignments, driver replacement, bulk/auto assign.
Journey execution	Loading stages, transit, GPS route points, proof of delivery, journey queries.
Queue dispatch	Queue organizations, FIFO driver queue, audit, room broadcasts.
Finance	Deposits (signed webhook), balances, commissions, tariffs, refunds.
Documents & media	Multipart upload, image re-encode, type/duplicate validation, history.
Notifications	Socket.IO `messages` (78 typed kinds), FCM push, SMTP e-mail, SMS.
Admin & governance	User/role ops, DB stats and maintenance, delinquency lifecycle, health.

3. Data flow design

3.1 Level 0 — context diagram

The system's external neighbours and the numbered flows between them. Every flow is an HTTPS/JSON or WSS contract except the GPS traces the driver app also keeps client-side; all inbound flows except the payment webhook and the public auth endpoints require a valid bearer token.

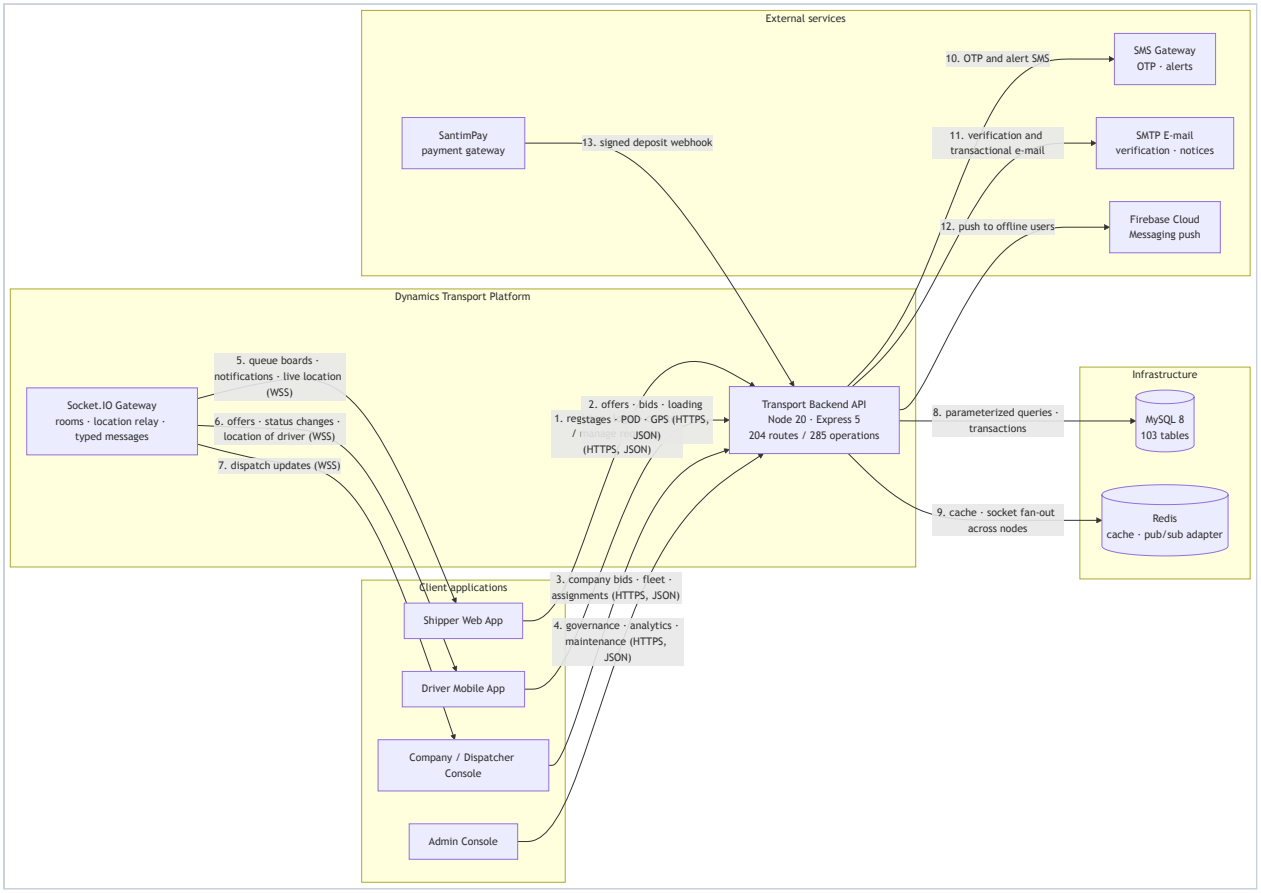


Figure 3.1 — Level 0: clients, platform, infrastructure and external services with numbered data flows

3.2 Sensitive-data entry points

Entry point	Channel	Lands in	Controls
Registration (name, phone, e-mail, password)	HTTPS JSON, public endpoint	`Users`, `usersCredential` (bcrypt hash)	Rate-limited auth routes, Joi validation, no password logging
OTP verification	SMS/e-mail out; HTTPS JSON in	Credential record (hashed OTP fields)	Single-use OTP, 24 h JWT issued only on success
Documents / images	multipart/form-data	`AttachedDocuments` + file store `/uploads`	Type & duplicate checks, image re-encode, access-controlled serving
GPS positions	WSS socket events + REST	`JourneyRoutePoints`, last-location store	Handshake JWT, room/ownership scoping, history retention

Entry point	Channel	Lands in	Controls
Deposit completion	Inbound signed webhook (public route)	`UserDeposit` → `UserBalance`	Signature verification, idempotent credit, transaction
Status transitions / cancellations	HTTPS JSON, authenticated	Status columns + decision/history rows	Role guards, transactional services, append-only audit

4. Component design

4.1 Layer contracts

- **Route** — path constant (from `Routes/EndPoints/*.js`) + middleware chain (token → validator → guard) + controller. No business logic.
- **Controller** — reads body/params/query and the authenticated user, invokes exactly one service, returns via `ServerResponder`.
- **Service** — owns rules and state transitions; throws `AppError` on violations; wraps multi-statement work in the transaction executor; writes history records; triggers notifications.
- **CRUD** — parameterized SQL helpers only; no business decisions.

4.2 Package layout

```
Routes/            HTTP surface — index.js mount table, EndPoints/*.js constants,
                   nested finance|company|queue|auth routers
Controllers/       request parsing · service invocation · ServerResponder
Services/          business logic (ShipperRequest/{create,read,update,cancellation,
                   reconcileStatus}, DriverRequest/journeyManagement, JourneyStatus,
                   UserDeposit, History, …)
CRUD/              Read/Update helpers (parameterized SQL)
Middleware/        VerifyToken · Validator (Joi) · GlobalErrorHandler · ScannerBlock
                   · CsrfOriginCheck · RequestId · RequestLogger · ownership guards
Utils/             ServerResponder · AppError · ListOfSeedData (status maps, roles,
                   cancellation reasons, 78-type message catalog) · DatabaseTransaction
                   · logger · WSPusher · Notifications · socket helpers
Config/            Express pipeline · CORS origins · Socket adapter · Redis ·
                   Firebase · Multer
E2ETests/          end-to-end suites + socket harness
docs/              product and API documentation (this package)
```

4.3 Cross-cutting components

Component	Design
`ServerResponder`	Single success renderer; derives the HTTP status from message/status fields when the caller does not pass one explicitly.
`AppError` + `globalErrorHandler`	Operational errors with HTTP codes; development sends the full error and stack, production masks 5xx and logs everything with request ids.
`Validator(schema, source)`	Joi middleware for body/query/params; reports all violations at once, strips unknown keys, guards empty POST bodies.
`ListOfSeedData`	Authoritative code-side seed: 20 journey statuses and their class sets (active/terminal/supporting/inactive), 12 roles, cancellation reasons, message catalog.
History service	`insertHistoryRecord` invoked before mutations; paired `*History` tables receive who/when/change-type rows.
Reconciler	`reconcileStatus` derives the correct status from decision evidence; whitelist = active statuses only.

Component	Design
Transaction context	Async-local transaction storage so nested service/CRUD calls join the ambient transaction instead of the pool.

4. Component design (continued)

4.4 The journey state machine

The behavioural core of the system. Statuses live in the `JourneyStatus` table (20 seeded rows); the id→name map and the class sets are defined once in `Utils/ListOfSeedData.js` and consumed by services, read paths and the reconciler. See the SRS §7 for the requirement-level rules; the design view is below.

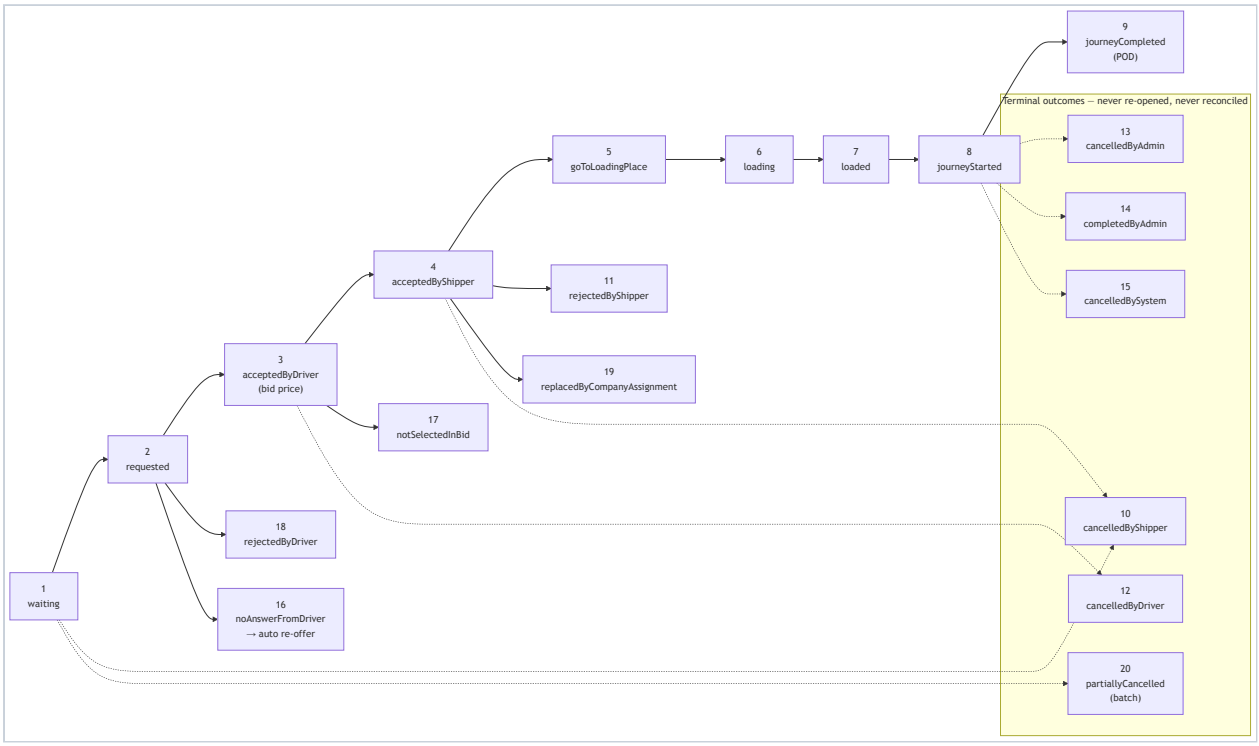


Figure 4.1 — Journey status machine: execution path (1→9), decision outcomes (11, 16–19) and terminal states (10, 12–15, 20)

4.5 Transition choreography (sequence)

Every transition follows the same five-step choreography; only the endpoint and the guard differ. Taking *“driver starts journey”* (8) as the example:

1. **Edge** — client calls `POST …/journeyStarted` with the bearer token; VerifyToken resolves the user, the Joi validator checks the payload.
2. **Guard** — the service loads the request row and its latest decision, verifies the caller's role/ownership and that the current status is in the allowed predecessor set (loaded = 7).
3. **Transaction** — inside one DB transaction: update `ShipperRequest.journeyStatusId`, create the `Journey` record (or update its start timestamp + GPS), append a `JourneyDecisions` row, append history.
4. **Commit** — the transaction commits; a non-throwing path returns the enriched request through `ServerResponder` (2xx envelope).
5. **Fan-out** — post-commit: shipper gets the typed `messages` notification, driver's app receives the ack, any reconciler cache for the row is invalidated.

> **Design invariant**
>
> Clients never write `journeyStatusId`. Every status change is produced by a service action with a guard, inside a transaction, with a decision row and a history row — which is what makes reconciliation (FR-04.6) and the audit trail possible.

4.6 Read-path enrichment

The detail read service assembles each row with its decisions, linked driver, batch and assignment context. Active company-target rows without an exact-status decision keep their own status and attach the available decisions instead of projecting to `waiting` — the projection bug this rule prevents is exactly the “row shows no connected driver” class of defect. The list read applies connected-first ordering with an `EXISTS(JourneyDecisions + DriverRequest)` subquery so connected rows sort first without row multiplication.

5. Data architecture

- **103 tables** in domain groups: identity, transport core, companies/fleet, queue, bidding, finance, documents, delinquency, ratings, subscriptions, reference data — detailed in the TDD package (`data-model.md`).
- **Core chain:** `ShipperRequest → DriverBid / DriverRequest → JourneyDecisions → Journey → JourneyRoutePoints / DeliveryConfirmations`, with status by `journeyStatusId` → `JourneyStatus`.
- **Keys:** UUID business keys (`*UniqueId`, indexed) in routes and payloads; numeric surrogate ids for status maps and joins.
- **Audit:** paired `*History` tables written through `insertHistoryRecord`; soft delete via `*DeletedAt`; append-only history.
- **Transactions:** `DatabaseTransaction` + async-local context; the executor selection honours the ambient transaction, else the pool — so service composition stays transaction-safe.
- **Hot paths:** paged reads on indexed sort keys; the connected-first ordering is a correlated `EXISTS` (no JOIN fan-out); Redis caches hot reads with TTL and is cache-through, not authoritative.

> **Reference data**
>
> Enumerations (journey statuses, roles, document types, payment statuses, cancellation reasons) are seeded rows joined on read and mirrored in `ListOfSeedData.js` so code and database stay aligned; tests assert the mirror.

6. API design

- **Surface:** 204 routes / 285 operations in 27 groups; path constants centralised in `Routes/EndPoints/*.js`; the mount table lives in `Routes/index.js` (plus nested finance/company/queue/auth routers). The complete inventory with auth labels ships as `endpoints.md`.
- **Envelopes:** success `{ message, ...data }`; error `{ status:"fail", message, error:{ statusCode, status, isOperational } }` with stacks dev-only.
- **Auth:** `Bearer <JWT>`; 17 public operations (auth/registration, health, payment webhook, maintenance); `jwtOrApiKey` variant accepts `x-api-key` for E2E bootstrapping on DB endpoints.
- **Validation:** Joi per route; **pagination:** `page/limit/sortBy/sortOrder`; **limits:** global + auth rate limiters; 1 MB body cap.
- **Versioning:** asset version 1.0.0; unversioned paths tracked by release tags and this document set.

> **Maintenance endpoints (observed)**
>
> A handful of admin database-maintenance routes are reachable without a token (auth commented out in source) — e.g. `POST /api/admin/createTable`, `DELETE /api/admin/dropTables`. This is documented as observed; in production they must remain network-restricted at the edge. `dropAllTables` and the dev OTP viewer do require token/API-key.

7. Real-time architecture

```
client A ── socket.io ──▶ node-1 ──┐
client B ── socket.io ──▶ node-2 ──┼── Redis pub/sub (Upstash, TLS)
client C ── socket.io ──▶ node-1 ──┘        │
                                   room fan-out:
                                   queueOrg:<id>[:date]
                                   per-socket direct emits
```

- **Handshake:** the client sends its JWT in `socket.handshake.auth`; init failures are answered with an error frame and the socket is disconnected.
- **Inbound events:** `queue:subscribe` / `queue:unsubscribe` (room join/leave with ack), `locationUpdateToDriver` / `locationUpdateToShipper` (position relay + persistence), `message` keep-alive (acked with `response`).
- **Outbound events:** `queue` board updates to rooms; typed notifications on the `messages` channel (78-kind catalog, default channel overrideable per call); location events including a replay of the last known position to newly connected sockets; structured error frames via `sendError`.
- **State:** connection store per user (for socket lookup and fallback routing), last-location store; users without a live socket fall back to FCM push / polling.
- **Robustness:** per-event try/catch → error frame (never crashes the loop); Redis adapter failures are logged and the process continues in single-server mode.

Room	Used for
`queueOrg:<orgUniqueId>`	All queue events for the organization (all dates).
`queueOrg:<orgUniqueId>:<date>`	Day-scoped dispatch: drivers after check-in, admins managing that day's queue.

8. Security architecture

Control	Implementation
Transport	TLS everywhere (Nginx); WSS for sockets; HSTS at the edge.
Authentication	bcrypt password + OTP second factor → JWT (24 h; payload userUniqueId, phone, role, iat/exp).
Authorization	Role middleware (12 roles) + ownership/company-scope guards; deny by default.
CORS	Explicit origin allow-list with credentials; no wildcard in production.
CSRF	Origin/Referer check for cookie-authenticated state-changing requests; bearer clients unaffected.
Headers & bots	helmet security headers; scanner/bot user-agent blocking.
Rate limiting	Global limiter + stricter auth-route limiter (429).

Control	Implementation
Input	Joi validation (strip unknown), 1 MB body cap, multipart limits, image re-encode, parameterized SQL only.
Fail-closed production checks	`App.js` refuses to boot production with development-only configuration (default secrets/debug) rather than starting insecurely.
Error leakage	Production 5xx responses masked; full detail logged server-side with request ids.
Secrets	Environment variables only (56 vars), never committed; payment webhook signature verified before crediting.

9. Error handling and observability

- **Uniform errors:** services throw `AppError(message, statusCode)`; the global handler renders the JSON envelope, logs 4xx as warnings and 5xx as errors with correlation (request id, method, path, user).
- **Logging:** structured Winston logs — request logger (timing, status), validator failures with field details, socket errors with stacks, application API errors. Client mistakes and server faults are logged in distinct classes so E2E results correlate to backend logs.
- **Health:** `GET /api/health` (liveness) and `GET /api/health/database` (DB reachability) — public, polled by monitoring; admin DB-stats endpoint for operators.
- **Performance:** query-performance middleware captures slow queries; request ids tie client reports to logs.

10. Deployment and environments

Aspect	Design
Process	PM2 (restart policy, log rotation); single entry bootstrapping HTTP + Socket.IO.
Edge	Nginx TLS terminator reverse-proxying to the app; WebSocket upgrade pass-through.
Environments	production `app.dynamicsroute.tech` · staging `dev.dynamicsroute.tech` · local (MAMP MySQL) — selected via `NODE_ENV` + env config; fail-closed checks in production.
Configuration	`.env` (DB, Redis, JWT secret, SMTP, FCM, payment keys) — 56 environment variables; not committed.
Data services	MySQL 8 (database `transport`, 103 tables); Redis/Upstash over TLS.
Scaling	Stateless REST → add nodes; Socket.IO Redis adapter keeps rooms consistent; PM2 cluster-capable.

11. Testing strategy

- **Unit/integration (jest):** 102 tests over services and utilities (status logic, validators, helpers).
- **End-to-end:** a 248-check suite covering registration → login → OTP → request lifecycle → bidding → assignment → journey → cancellation, plus negative probes (401/400/404) and a socket harness (room subscribe, board broadcast, location relay, typed messages).
- **Route contract checks:** the published endpoint inventory is verified against a running instance by live preflight probes — only routes that answer non-404 are listed, auth labels come from observed 401 behaviour.
- **Regression gate:** per-run baselines with named failure comparison (e.g. 239 passed / 3 known-failing / 6 skipped) so new breakage is distinguished from accepted debt.

12. Design — requirements traceability

SRS requirement group	This document	Primary code area
FR-01 identity & access	§6 API design · §8 Security	`Middleware/VerifyToken`, `Services/User/*`, `Routes/auth`
FR-02 shipper requests	§4 Component design · §5 Data	`Services/ShipperRequest/*`
FR-03 bidding & assignment	§4.3–§4.5	`Services/DriverRequest/*`, company bid/assignment services
FR-04 journey execution	§4.4–§4.6 (state machine, choreography)	`Services/JourneyStatus`, `journeyManagement`, POD services
FR-05 company & fleet	§5 Data architecture	`Services/*Company*`, `VehicleDriver.service`
FR-06 dispatch queue	§7 Real-time architecture	`Services/Queue/*`, `Config/SocketAdapter`
FR-07 finance	§3.2 entry points · §8 Security	`Services/UserDeposit/*`, commission services
FR-08 documents	§3.2 · §4.2	`Middleware/*Document*`, upload controllers
FR-09 notifications	§7 Real-time architecture	`Utils/Notifications`, `MessageTypes`, FCM
FR-10..12 admin / ratings / subscriptions	§9–§10	`Services/Admin/*`, reference services
NFR-01..08	§8 Security · §9 · §10 · §11	`Config/Express.config`, `App.js` fail-closed checks, suites

Appendix A — document reproduction pipeline

The four submission files are generated from the sources in this folder:

```
docs/INSA/
├── sources/
│   ├── srs.html · sdd.html        HTML sources (print-first CSS)
│   ├── assets/*.mmd               mermaid diagram sources (3 figures)
│   ├── assets/*.svg               rendered diagrams (mmdc)
│   ├── request-response/          endpoint inventory + README + 6 captures
│   └── tdd/                       TDD/LLD documents + shared captures
├── build.sh                       render PDFs (headless Chrome) + build ZIPs
├── SRS.pdf · SDD.pdf              uploads #2 and #3
├── RequestResponse.zip            upload #1
└── API-Technical-Specifications-TDD-LLD.zip   upload #4
```

- Diagrams: `mmdc -i <name>.mmd -o <name>.svg -w 1600` (mermaid-cli; falls back to the system Chrome via `PUPPETEER_EXECUTABLE_PATH`).
- PDFs: headless Chrome `--headless --print-to-pdf --no-pdf-header-footer` over the HTML sources; PDF Title metadata comes from the HTML `<title>`.
- ZIPs: plain `zip -qr` over the source folders (inventory + captures / TDD documents + captures).
- Verification: `pdfinfo`, `pdftotext`, `unzip -t`, PII scan over the source tree (see the package README checklist).

End of SDD — Transport Management System API v1.0.0 · Asset: transport · Prepared for INSA asset registration.